

Smart Campus Management System

CS-104L Object-Oriented Programming

HITEC University Taxila

Group Members

- Ahmad Ali | Reg No: 25-CS-067
- Umer Altaf | Reg No: 25-CS-057
- Muhammed Ahmad | Reg No: 25-CS-252

Introduction

This project is a simple command-line Smart Campus Management System written in C++. It manages basic campus records such as students, faculty, courses, library items, fee records, hostel rooms, and reports. The code is made with beginner-level OOP concepts and simple arrays so it can be understood and explained by second-semester students.

System Modules

- Person module: Person, Student, GradStudent, Faculty, and Staff classes.
- Course module: Course and Enrollment classes with capacity checking.
- Library module: Book and Journal catalog using file handling and arrays.
- Finance module: FeeRecord and Invoice classes with copy handling and static invoice counter.
- Hostel module: Room, HostelBlock, and HostelManager using composition and multiple inheritance.
- Reports module: student sorting and campus text report generation.

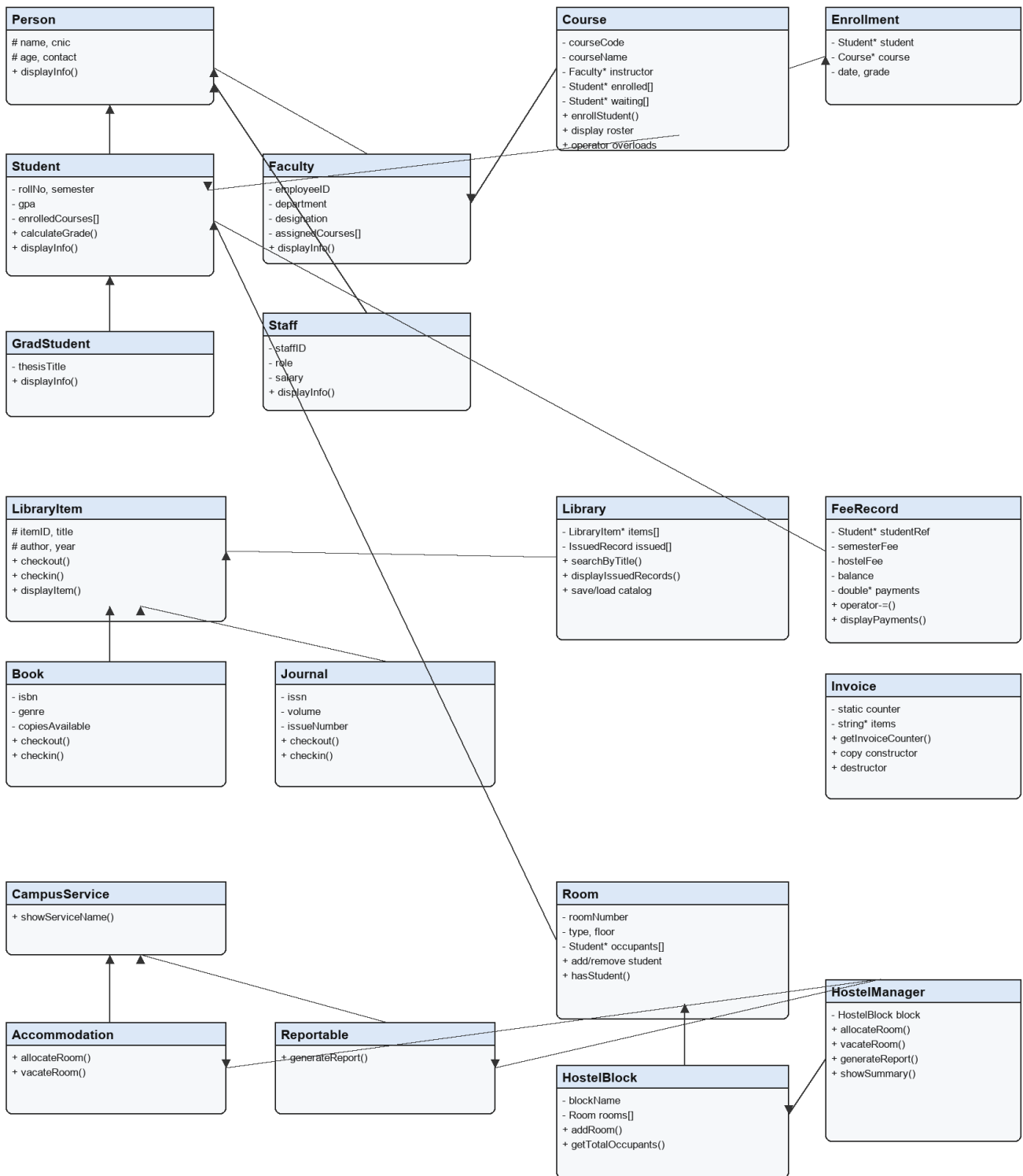
Class Diagram

The class diagram below shows the main classes and relationships used in the project.

Smart Campus Management System (SCMS)

Smart Campus Management System - Class Diagram

HITEC University Taxila | CS-104L OOP



Legend: arrows show inheritance or main object relationships. See docs/class_diagram.mmd for Mermaid source.

Smart Campus Management System (SCMS)

OOP Concepts Implemented

- Classes and Objects: All modules use C++ classes and objects.
- Encapsulation: Private data members are accessed through public methods.
- Constructors: Default and parameterized constructors are used in many classes.
- Destructors: Library and Invoice clean dynamic memory.
- Single Inheritance: Student inherits from Person.
- Multi-level Inheritance: GradStudent inherits from Student.
- Multiple Inheritance: HostelManager inherits from Accommodation and Reportable.
- Virtual Inheritance: Accommodation and Reportable virtually inherit CampusService.
- Abstract Classes: Person, LibraryItem, Accommodation, and Reportable.
- Runtime Polymorphism: Person pointers call different displayInfo functions.
- Operator Overloading: Course ==, Course <<, Course +, FeeRecord -=.
- Friend Functions: operator<< is used for Course and Invoice.
- Static Members: Invoice uses invoiceCounter.
- Copy Constructor and Assignment: FeeRecord and Invoice examples.
- Search Functions: Library has searchByTitle and searchByID.
- Array Collections: Arrays are used for courses, library items, rooms, and students.
- Exception Handling: CapacityExceededException and OverdueException.
- File I/O: Library catalog and campus report files.
- Sorting: Reports sort students by GPA.
- Composition: HostelManager contains HostelBlock.
- Aggregation: Course stores Faculty pointer and Room stores Student pointers.

Library and Finance Details

The Library module loads records from data/library_catalog.txt, displays books and journals, searches by title or ID, issues a book, shows issued records, and checks overdue returns. The Finance module tracks fee balance, accepts payment with operator-=, deep-copies payment history, and creates invoices with a static invoice counter.

Testing Summary

- Build command .\build.bat completed successfully.
- Menu options 1 to 7 were tested.
- Library demo loaded catalog data, issued B001, and showed overdue fine.
- Finance demo showed payment, copy constructor, copy assignment, invoice, and invoice copy.
- Hostel demo showed service name, allocation, duplicate check, summary, report, and vacate room.
- Reports demo sorted students by GPA and created data/campus_report.txt.
- Wrong input test with abc did not freeze the program.

Challenges and Solutions

- Understanding polymorphism: solved by using Person* array in main.cpp.
- Managing dynamic memory: solved by deleting Library items and Invoice item arrays.
- Handling full course capacity: solved by using a custom exception class.

Smart Campus Management System (SCMS)

- File handling format: solved by using simple pipe-separated text data.

Conclusion

This project helped practice core C++ OOP concepts in a practical campus system. The project is intentionally console-based and simple so each module can be explained in viva. Before final submission, real output screenshots should be added by the group.

References

- Course project brief provided by HITEC University Taxila.
- C++ class notes and lab practice.
- cppreference.com for basic C++ syntax reference.